DBDS
MINI
PROJECT

BY

NITHISH TONY WS
23AM158
312323247077
II YEAR AML - B

Jumble Word Game with MySQL Database

AIM

by using mysql.connector & Tkinter ,
by accessing the database named "world" to create a Jumble Word Game like interface which question based on database data's and answer fetch and check through database

Jumble Word Game with MySQL Database - Algorithm

1. Initialization Phase

1. Initialize Application
   - Create main Tkinter window
   - Configure window properties (title, size)
   - Set up initial variables (score = 0, total_questions = 0)

2. Database Connection
   - Establish MySQL connection to "world" database
   - Create cursor object for database operations
   - Handle potential connection errors

3. UI Component Setup
   - Create question display label
   - Create answer input field
   - Create submit button
   - Create score display label
   - Set up event handlers

2. Question Generation

1. Random Question Selection
   - Generate SQL query to select random city-country pair from database
   - Execute query against "world" database
   - Retrieve result set

2. Question Formatting
   - Format database result into user-friendly question text
   - Example: "Which country is the city 'Paris' located in?"
   - Update question label with formatted text
   - Increment total_questions counter

3. User Interaction

1. User Input Handling
   - Wait for user to enter answer in input field
   - Capture submit button click event
   - Retrieve and trim user's answer

2. Answer Validation
   - Compare user's answer with correct answer from database (case-insensitive)
   - Determine if answer is correct or incorrect

4. Feedback Provision

1. For Correct Answers
   - Display "Correct!" message
   - Increment score counter
   - Update score display

2. For Incorrect Answers
   - Display "Incorrect" message
   - Show correct answer from database
   - Update score display

5. Quiz Progression

1. Score Tracking
   - Maintain running score (correct_answers/total_questions)
   - Update score display after each question

2. Question Cycling
   - Clear previous answer input
   - Generate new random question from database
   - Update question display

6. Termination

1. Quiz Completion
   - Option for user to quit or continue with more questions
   - Display final score upon completion

2. Cleanup
   - Close database cursor
   - Close database connection
   - Destroy Tkinter window

7. Error Handling

1. Database Connection Errors
   - Catch and display database connection errors
   - Provide user-friendly error messages

2. Query Execution Errors
   - Handle potential SQL query errors
   - Provide fallback questions if database access fails

8. Helper Functions

1. load_next_question()
   - Generates and displays the next question
   - Updates question counter

2. check_answer()
   - Validates user input against database record
   - Updates score and provides feedback
   - Triggers next question

PROGRAM - SOURCE CODE

```
import mysql.connector
import tkinter as tk
from tkinter import messagebox, ttk
import random

class JumbleWordApp:
    def __init__(self, root):
        self.root = root
        self.root.title("World Database Jumble
Word Game")
        self.root.geometry("600x500")
        self.root.configure(bg="#f0f0f0")

        # Database connection
        self.connection = None
        self.cursor = None
        self.connect_to_database()

        # Game variables
```

```
        self.current_word = ""
        self.current_jumbled = ""
        self.current_hint = ""
        self.current_category = ""
        self.score = 0
        self.questions_asked = 0

        # UI Elements
        self.create_widgets()

        # Start the game
        self.new_word()

    def connect_to_database(self):
        try:
            self.connection =
mysql.connector.connect(
                host="localhost",
                user="root",      # Replace with your
MySQL username
```

```
                password="root",      # Replace with
your MySQL password
                database="world"
            )
            self.cursor = self.connection.cursor()
            print("Connected to database
successfully!")
        except mysql.connector.Error as err:
            messagebox.showerror("Database
Error", f"Could not connect to database:
{err}")
            self.root.destroy()

    def create_widgets(self):
        # Top frame for game info
        top_frame = tk.Frame(self.root,
bg="#f0f0f0")
        top_frame.pack(pady=20)

        tk.Label(top_frame, text="World
Geography Jumble Word Game",
```

```
                font=("Arial", 16, "bold"),
bg="#f0f0f0").pack()

        # Score display
        self.score_label = tk.Label(top_frame,
text="Score: 0",
                                font=("Arial", 12),
bg="#f0f0f0")
        self.score_label.pack(pady=5)

        # Game content frame
        game_frame = tk.Frame(self.root,
bg="#f0f0f0")
        game_frame.pack(pady=10,
fill=tk.BOTH, expand=True)

        # Category label
        self.category_label =
tk.Label(game_frame, text="Category: None",
                                 font=("Arial", 12,
"italic"), bg="#f0f0f0")
```

```
        self.category_label.pack(pady=10)

        # Jumbled word display
        self.jumbled_label =
tk.Label(game_frame, text="",
                                font=("Arial", 18,
"bold"), bg="#f0f0f0")
        self.jumbled_label.pack(pady=10)

        # Hint display
        self.hint_label = tk.Label(game_frame,
text="Hint: None",
                               font=("Arial", 10),
bg="#f0f0f0", wraplength=500)
        self.hint_label.pack(pady=10)

        # Answer entry
        tk.Label(game_frame, text="Your
Answer:",
                 font=("Arial", 12),
bg="#f0f0f0").pack(pady=5)
```

```
        self.answer_entry =
tk.Entry(game_frame, font=("Arial", 14),
width=30)
        self.answer_entry.pack(pady=10)
        self.answer_entry.bind("<Return>",
lambda event: self.check_answer())

        # Buttons frame
        button_frame = tk.Frame(self.root,
bg="#f0f0f0")
        button_frame.pack(pady=20)

        self.submit_btn =
tk.Button(button_frame, text="Submit
Answer",

command=self.check_answer,
                                font=("Arial", 12),
bg="#4CAF50", fg="white",
                                padx=10, pady=5)
```

```
        self.submit_btn.pack(side=tk.LEFT,
padx=10)

        self.next_btn = tk.Button(button_frame,
text="Next Word",

command=self.new_word,
                                font=("Arial", 12),
bg="#2196F3", fg="white",
                                padx=10, pady=5)
        self.next_btn.pack(side=tk.LEFT,
padx=10)

        self.hint_btn = tk.Button(button_frame,
text="Show Hint",

command=self.show_hint,
                                font=("Arial", 12),
bg="#FFC107", fg="black",
                                padx=10, pady=5)
```

```
        self.hint_btn.pack(side=tk.LEFT,
padx=10)

    def get_random_word_from_db(self):
        """Fetch a random word and hint from
the database"""
        # List of possible tables and columns to
query
        tables_columns = [
            ("country", "Name", "Region", "Cities
in this country"),
            ("country", "Name", "Continent",
"Located in this continent"),
            ("city", "Name", "CountryCode", "City
in this country"),
            ("countrylanguage", "Language",
"CountryCode", "Language spoken in this
country")
        ]

        # Randomly choose a table and columns
```

```
        table, column, hint_column, hint_prefix =
random.choice(tables_columns)

        # Get a random row from the selected
table
        try:
            if table == "country":
                query = f"SELECT {column},
{hint_column} FROM {table} ORDER BY
RAND() LIMIT 1"
            elif table == "city":
                query = f"""
                    SELECT c.{column}, co.Name
                    FROM {table} c
                    JOIN country co ON
c.CountryCode = co.Code
                    ORDER BY RAND() LIMIT 1
                """
            elif table == "countrylanguage":
                query = f"""
                    SELECT cl.{column}, co.Name
```

```
                FROM {table} cl
                JOIN country co ON
cl.CountryCode = co.Code
                ORDER BY RAND() LIMIT 1
            """

        self.cursor.execute(query)
        result = self.cursor.fetchone()

        if result:
            word = result[0]
            hint_value = result[1]

            # Skip words that are too short or
have special characters
            if len(word) < 4 or not
word.isalpha():
                return
self.get_random_word_from_db()

            # Generate hint
```

```
            hint = f"{hint_prefix}: {hint_value}"

            return word, hint, table
        else:
            return
self.get_random_word_from_db()

    except mysql.connector.Error as err:
        messagebox.showerror("Database
Error", f"Error fetching word: {err}")
        return "ERROR", "Database error
occurred", "error"

  def jumble_word(self, word):
    """Create a jumbled version of the
word"""
    word_list = list(word.upper())
    random.shuffle(word_list)
    return ".join(word_list)

  def new_word(self):
```

```
        """Get a new word and update the UI"""
        self.current_word, self.current_hint,
self.current_category =
self.get_random_word_from_db()
        self.current_jumbled =
self.jumble_word(self.current_word)

        # Update UI

self.jumbled_label.config(text=self.current_ju
mbled)

self.category_label.config(text=f"Category:
{self.current_category.capitalize()}")
        self.hint_label.config(text="Hint: (Click
'Show Hint' to reveal)")
        self.answer_entry.delete(0, tk.END)
        self.answer_entry.focus()

    def show_hint(self):
        """Display the hint for the current word"""
```

```
        self.hint_label.config(text=f"Hint:
{self.current_hint}")

    def check_answer(self):
        """Check if the user's answer is
correct"""
        user_answer =
self.answer_entry.get().strip().upper()
        correct_answer =
self.current_word.upper()

        if user_answer == correct_answer:
            self.score += 10
            self.score_label.config(text=f"Score:
{self.score}")
            messagebox.showinfo("Correct!",
"That's right! +10 points")
            self.new_word()
        else:
            messagebox.showinfo("Incorrect",
f"Sorry, that's not right. Try again or click 'Next
```

```
Word'\nThe correct answer was:
{self.current_word}")

    def on_closing(self):
        """Close database connection and
destroy root window"""
        if self.connection:
            self.connection.close()
        self.root.destroy()

if __name__ == "__main__":
    root = tk.Tk()
    app = JumbleWordApp(root)
    root.protocol("WM_DELETE_WINDOW",
app.on_closing)
    root.mainloop()
```

jumbled.py - C:\Users\Nerub\OneDrive\Desktop\jumbled.py (3.12.9)

File Edit Format Run Options Window Help

```python
import mysql.connector
import tkinter as tk
from tkinter import messagebox, ttk
import random

class JumbleWordApp:
    def __init__(self, root):
        self.root = root
        self.root.title("World Database Jumble Word Game")
        self.root.geometry("600x500")
        self.root.configure(bg="#f0f0f0")

        # Database connection
        self.connection = None
        self.cursor = None
        self.connect_to_database()

        # Game variables
        self.current_word = ""
        self.current_jumbled = ""
        self.current_hint = ""
        self.current_category = ""
        self.score = 0
        self.questions_asked = 0

        # UI Elements
        self.create_widgets()

        # Start the game
        self.new_word()

    def connect_to_database(self):
        try:
            self.connection = mysql.connector.connect(
                host="localhost",
                user="root",      # Replace with your MySQL username
                password="root",      # Replace with your MySQL password
                database="world"
            )
            self.cursor = self.connection.cursor()
            print("Connected to database successfully!")
```

jumbled.py - C:\Users\Nerub\OneDrive\Desktop\jumbled.py (3.12.9)

File Edit Format Run Options Window Help

```
        print("Connected to database successfully.")
    except mysql.connector.Error as err:
        messagebox.showerror("Database Error", f"Could not connect to database: {err}")
        self.root.destroy()

def create_widgets(self):
    # Top frame for game info
    top_frame = tk.Frame(self.root, bg="#f0f0f0")
    top_frame.pack(pady=20)

    tk.Label(top_frame, text="World Geography Jumble Word Game",
             font=("Arial", 16, "bold"), bg="#f0f0f0").pack()

    # Score display
    self.score_label = tk.Label(top_frame, text="Score: 0",
                                font=("Arial", 12), bg="#f0f0f0")
    self.score_label.pack(pady=5)

    # Game content frame
    game_frame = tk.Frame(self.root, bg="#f0f0f0")
    game_frame.pack(pady=10, fill=tk.BOTH, expand=True)

    # Category label
    self.category_label = tk.Label(game_frame, text="Category: None",
                                   font=("Arial", 12, "italic"), bg="#f0f0f0")
    self.category_label.pack(pady=10)

    # Jumbled word display
    self.jumbled_label = tk.Label(game_frame, text="",
                                  font=("Arial", 18, "bold"), bg="#f0f0f0")
    self.jumbled_label.pack(pady=10)

    # Hint display
    self.hint_label = tk.Label(game_frame, text="Hint: None",
                               font=("Arial", 10), bg="#f0f0f0", wraplength=500)
    self.hint_label.pack(pady=10)

    # Answer entry
    tk.Label(game_frame, text="Your Answer:",
             font=("Arial", 12), bg="#f0f0f0").pack(pady=5)

    self.answer_entry = tk.Entry(game_frame, font=("Arial", 14), width=30)
```

jumbled.py - C:\Users\Nerub\OneDrive\Desktop\jumbled.py (3.12.9)

File Edit Format Run Options Window Help

```python
        self.answer_entry = tk.Entry(game_frame, font=("Arial", 14), width=30)
        self.answer_entry.pack(pady=10)
        self.answer_entry.bind("<Return>", lambda event: self.check_answer())

        # Buttons frame
        button_frame = tk.Frame(self.root, bg="#f0f0f0")
        button_frame.pack(pady=20)

        self.submit_btn = tk.Button(button_frame, text="Submit Answer",
                                    command=self.check_answer,
                                    font=("Arial", 12), bg="#4CAF50", fg="white",
                                    padx=10, pady=5)
        self.submit_btn.pack(side=tk.LEFT, padx=10)

        self.next_btn = tk.Button(button_frame, text="Next Word",
                                  command=self.new_word,
                                  font=("Arial", 12), bg="#2196F3", fg="white",
                                  padx=10, pady=5)
        self.next_btn.pack(side=tk.LEFT, padx=10)

        self.hint_btn = tk.Button(button_frame, text="Show Hint",
                                  command=self.show_hint,
                                  font=("Arial", 12), bg="#FFC107", fg="black",
                                  padx=10, pady=5)
        self.hint_btn.pack(side=tk.LEFT, padx=10)

    def get_random_word_from_db(self):
        """Fetch a random word and hint from the database"""
        # List of possible tables and columns to query
        tables_columns = [
            ("country", "Name", "Region", "Cities in this country"),
            ("country", "Name", "Continent", "Located in this continent"),
            ("city", "Name", "CountryCode", "City in this country"),
            ("countrylanguage", "Language", "CountryCode", "Language spoken in this country")
        ]

        # Randomly choose a table and columns
        table, column, hint_column, hint_prefix = random.choice(tables_columns)

        # Get a random row from the selected table
        try:
```

jumbled.py - C:\Users\Nerub\OneDrive\Desktop\jumbled.py (3.12.9)

File Edit Format Run Options Window Help

```python
        if table == "country":
            query = f"SELECT {column}, {hint_column} FROM {table} ORDER BY RAND() LIMIT 1"
        elif table == "city":
            query = f"""
                SELECT c.{column}, co.Name
                FROM {table} c
                JOIN country co ON c.CountryCode = co.Code
                ORDER BY RAND() LIMIT 1
            """
        elif table == "countrylanguage":
            query = f"""
                SELECT cl.{column}, co.Name
                FROM {table} cl
                JOIN country co ON cl.CountryCode = co.Code
                ORDER BY RAND() LIMIT 1
            """

        self.cursor.execute(query)
        result = self.cursor.fetchone()

        if result:
            word = result[0]
            hint_value = result[1]

            # Skip words that are too short or have special characters
            if len(word) < 4 or not word.isalpha():
                return self.get_random_word_from_db()

            # Generate hint
            hint = f"{hint_prefix}: {hint_value}"

            return word, hint, table
        else:
            return self.get_random_word_from_db()

    except mysql.connector.Error as err:
        messagebox.showerror("Database Error", f"Error fetching word: {err}")
        return "ERROR", "Database error occurred", "error"
```

jumbled.py - C:\Users\Nerub\OneDrive\Desktop\jumbled.py (3.12.9)

File Edit Format Run Options Window Help

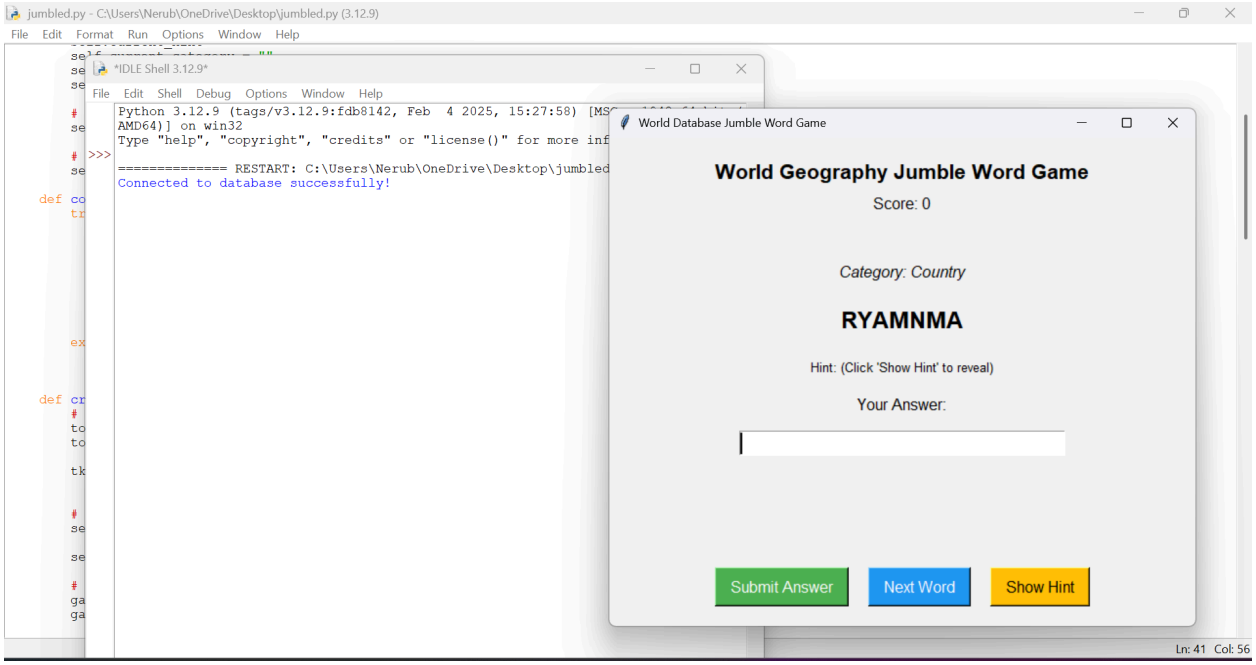
```python
def jumble_word(self, word):
    """Create a jumbled version of the word"""
    word_list = list(word.upper())
    random.shuffle(word_list)
    return ''.join(word_list)

def new_word(self):
    """Get a new word and update the UI"""
    self.current_word, self.current_hint, self.current_category = self.get_random_word_from_db()
    self.current_jumbled = self.jumble_word(self.current_word)

    # Update UI
    self.jumbled_label.config(text=self.current_jumbled)
    self.category_label.config(text=f"Category: {self.current_category.capitalize()}")
    self.hint_label.config(text="Hint: (Click 'Show Hint' to reveal)")
    self.answer_entry.delete(0, tk.END)
    self.answer_entry.focus()

def show_hint(self):
    """Display the hint for the current word"""
    self.hint_label.config(text=f"Hint: {self.current_hint}")

def check_answer(self):
    """Check if the user's answer is correct"""
    user_answer = self.answer_entry.get().strip().upper()
    correct_answer = self.current_word.upper()

    if user_answer == correct_answer:
        self.score += 10
        self.score_label.config(text=f"Score: {self.score}")
        messagebox.showinfo("Correct!", "That's right! +10 points")
        self.new_word()
    else:
        messagebox.showinfo("Incorrect", f"Sorry, that's not right. Try again or click 'Next Word'\nThe correct answer was: {self.current_word}")

def on_closing(self):
    """Close database connection and destroy root window"""
    if self.connection:
        self.connection.close()
    self.root.destroy()
```

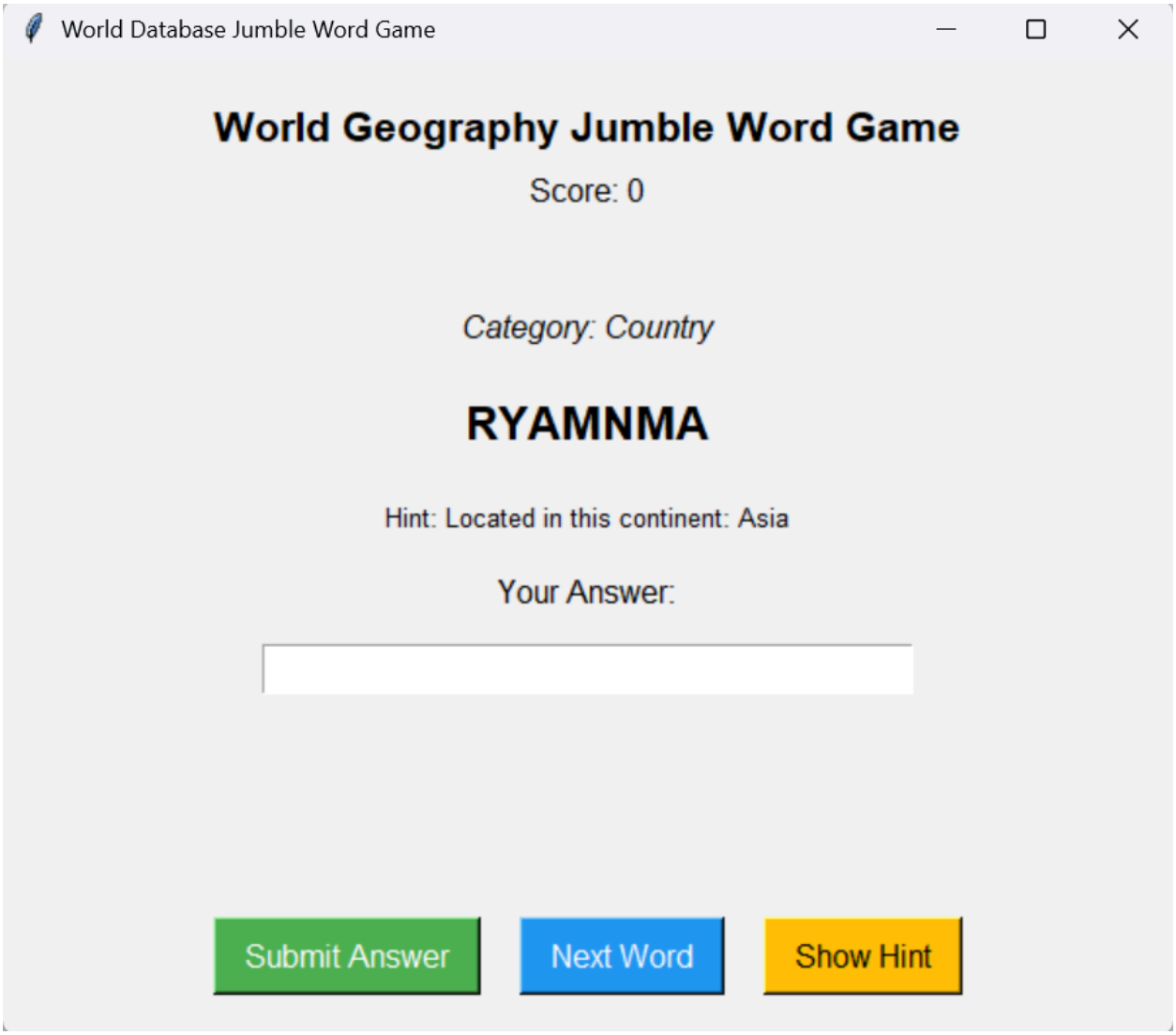
jumbled.py - C:\Users\Nerub\OneDrive\Desktop\jumbled.py (3.12.9)

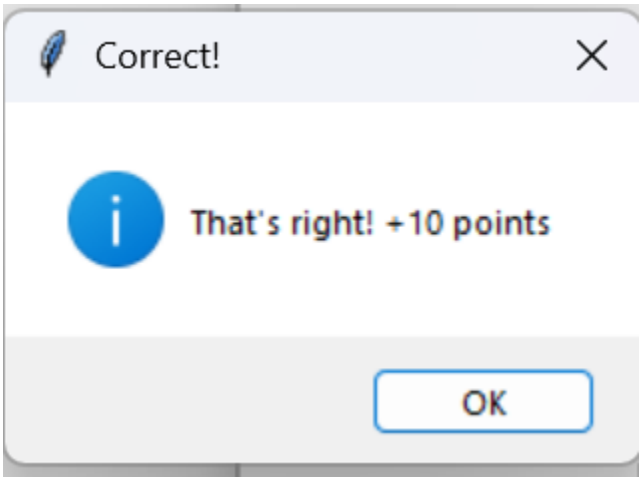
File Edit Format Run Options Window Help

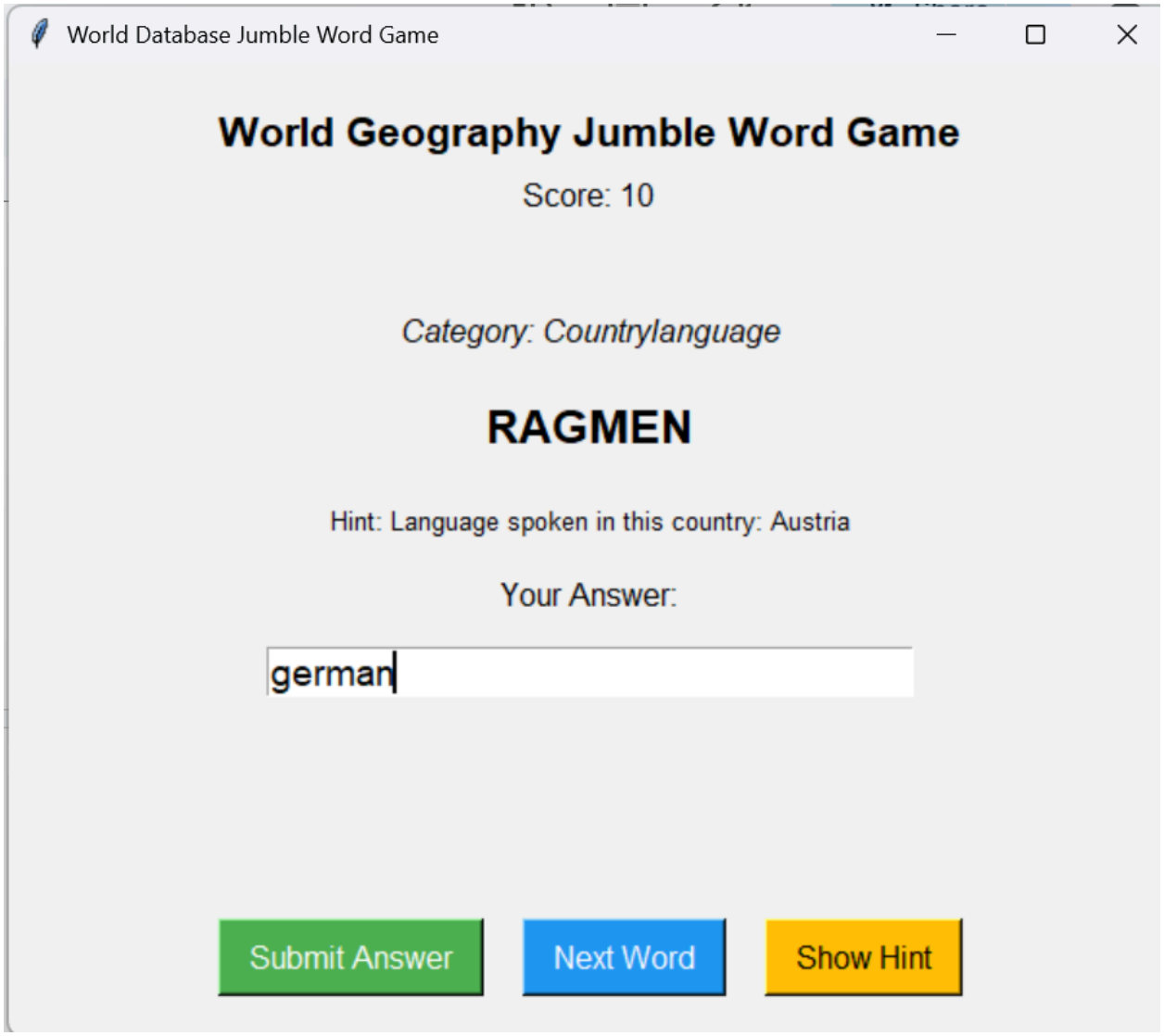
```python
    def new_word(self):
        """Get a new word and update the UI"""
        self.current_word, self.current_hint, self.current_category = self.get_random_word_from_db()
        self.current_jumbled = self.jumble_word(self.current_word)

        # Update UI
        self.jumbled_label.config(text=self.current_jumbled)
        self.category_label.config(text=f"Category: {self.current_category.capitalize()}")
        self.hint_label.config(text="Hint: (Click 'Show Hint' to reveal)")
        self.answer_entry.delete(0, tk.END)
        self.answer_entry.focus()

    def show_hint(self):
        """Display the hint for the current word"""
        self.hint_label.config(text=f"Hint: {self.current_hint}")

    def check_answer(self):
        """Check if the user's answer is correct"""
        user_answer = self.answer_entry.get().strip().upper()
        correct_answer = self.current_word.upper()

        if user_answer == correct_answer:
            self.score += 10
            self.score_label.config(text=f"Score: {self.score}")
            messagebox.showinfo("Correct!", "That's right! +10 points")
            self.new_word()
        else:
            messagebox.showinfo("Incorrect", f"Sorry, that's not right. Try again or click 'Next Word'\nThe correct answer was: {self.current_word}")

    def on_closing(self):
        """Close database connection and destroy root window"""
        if self.connection:
            self.connection.close()
        self.root.destroy()

if __name__ == "__main__":
    root = tk.Tk()
    app = JumbleWordApp(root)
    root.protocol("WM_DELETE_WINDOW", app.on_closing)
    root.mainloop()
```

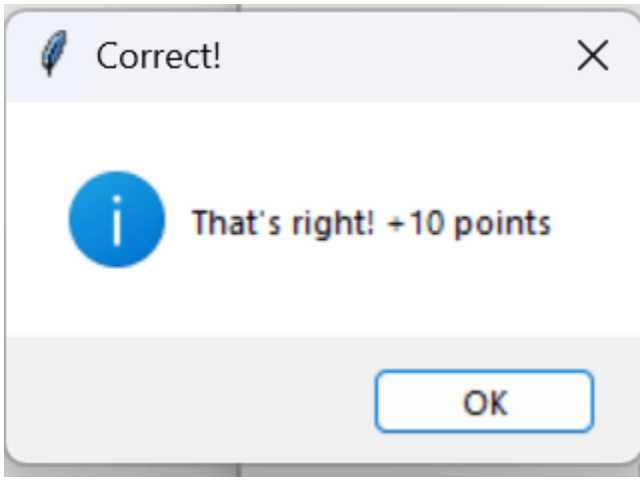
OUTPUT - GUI

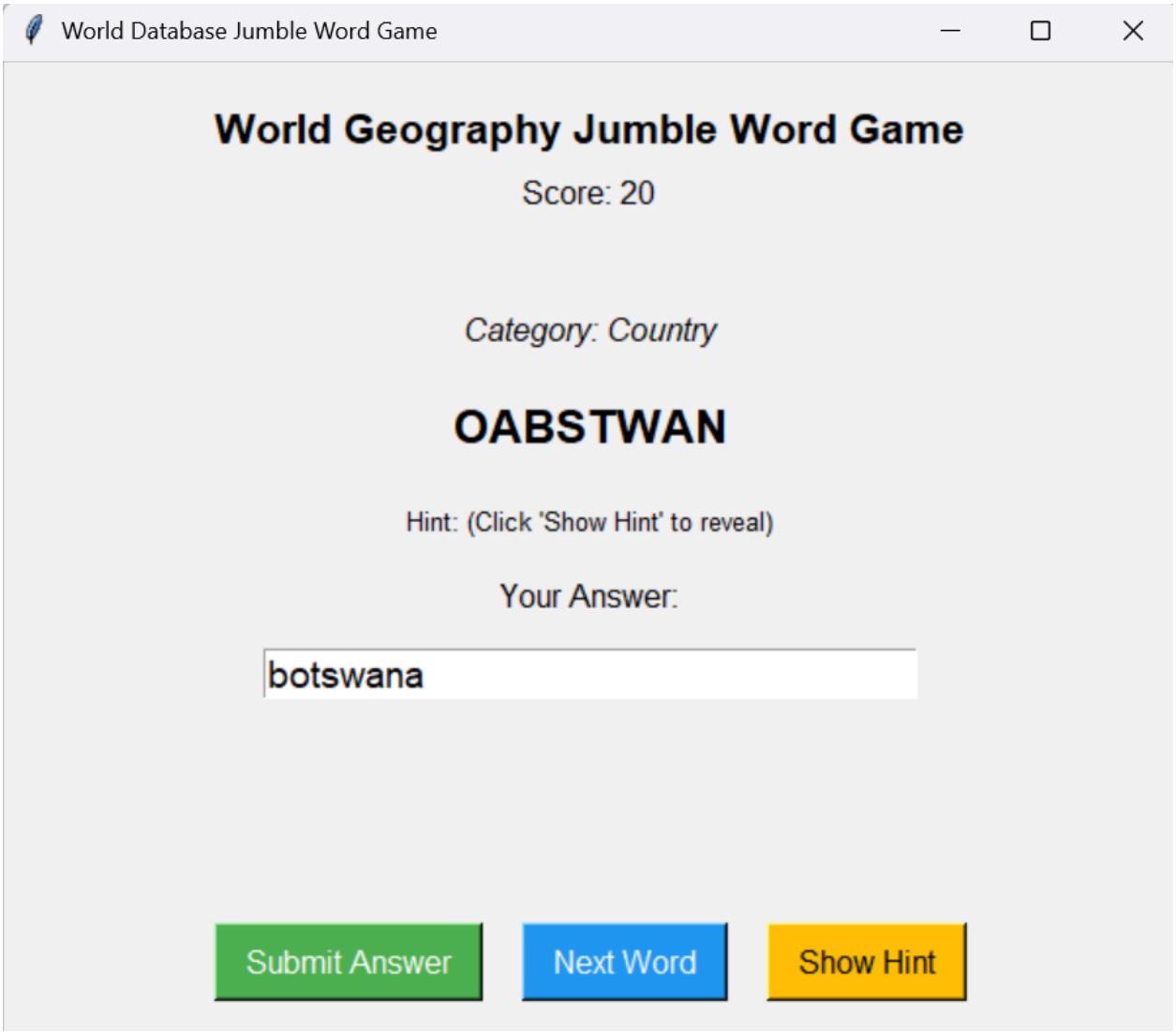

jumbled.py - C:\Users\Nerub\OneDrive\Desktop\jumbled.py (3.12.9)
File Edit Format Run Options Window Help
IDLE Shell 3.12.9
File Edit Shell Debug Options Window Help
Python 3.12.9 (tags/v3.12.9:fdb8142, Feb 4 2025, 15:27:58)
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more inf
>>>
========== RESTART: C:\Users\Nerub\OneDrive\Desktop\jumbled
Connected to database successfully!
World Database Jumble Word Game
World Geography Jumble Word Game
Score: 0
Category: Country
RYAMNMA
Hint: (Click 'Show Hint' to reveal)
Your Answer:
Submit Answer
Next Word
Show Hint
Ln: 41 Col: 56

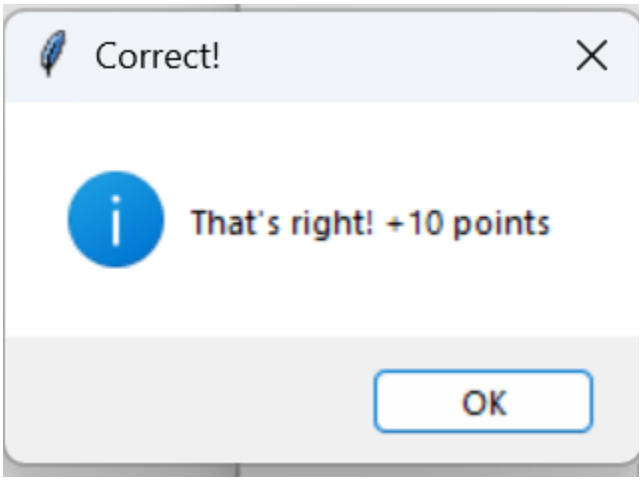

World Database Jumble Word Game
World Geography Jumble Word Game
Score: 0
Category: Country
RYAMNMA
Hint: Located in this continent: Asia
Your Answer:
Submit Answer
Next Word
Show Hint

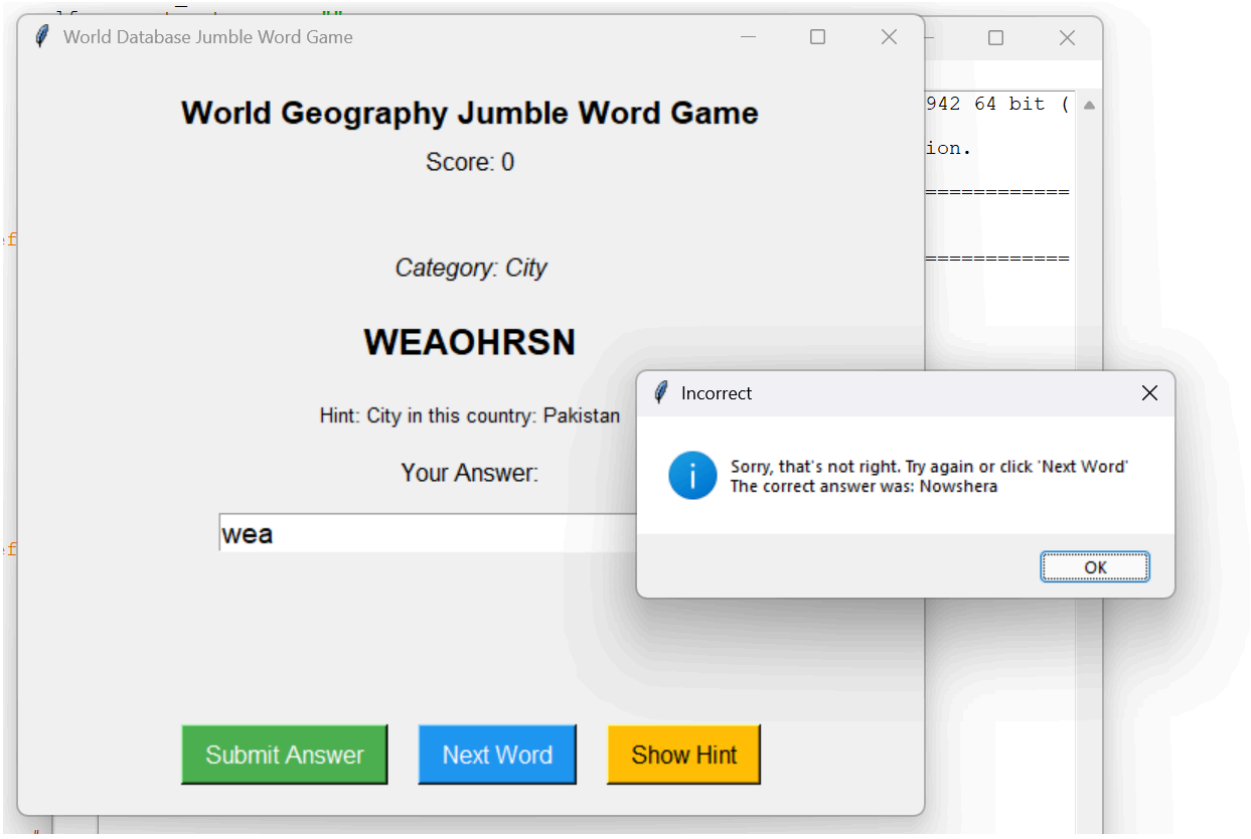

Correct!
That's right! +10 points
OK


World Database Jumble Word Game
World Geography Jumble Word Game
Score: 10
Category: Countrylanguage
RAGMEN
Hint: Language spoken in this country: Austria
Your Answer:
german
Submit Answer
Next Word
Show Hint


Correct!
That's right! +10 points
OK


World Database Jumble Word Game
World Geography Jumble Word Game
Score: 20
Category: Country
OABSTWAN
Hint: (Click 'Show Hint' to reveal)
Your Answer:
botswana
Submit Answer
Next Word
Show Hint


Correct!
That's right! +10 points
OK


World Database Jumble Word Game
World Geography Jumble Word Game
Score: 0
Category: City
WEAOHRSN
Hint: City in this country: Pakistan
Your Answer:
wea
Submit Answer
Next Word
Show Hint
Incorrect
Sorry, that's not right. Try again or click 'Next Word'
The correct answer was: Nowshera
OK

CONCLUSION

The Jumble Word Game with MySQL Database demonstrates how educational software can leverage existing data sources to create engaging learning experiences. By combining database access with interactive user interfaces, we've created a tool that makes geographical knowledge accessible and enjoyable while showcasing good software design principles. The modular architecture ensures the application can grow and adapt to future educational needs.